

Tool Calling

function

What is tool calling?



- **Tool calling** (also known as **function calling**) is a capability in large language models (LLMs) that enables the model to **request the execution of external functions or APIs** by outputting structured, machine-readable requests for specific tools, along with the necessary arguments, rather than generating a natural language response
- Components
  - Tool Definition
  - Tool Schema and binding
  - Tool call
  - Tool Execution
  - Result Integration

Tool

- **A tool** is an external function, API, or capability that an AI model can use to interact with the real world, perform computations, retrieve information, or take actions beyond its inherent capabilities
- **A tool** is a callable function or resource that is exposed to a language model through a structured interface, allowing the model to request its execution to accomplish tasks that require specialized capabilities, real-time data, or interaction with external systems
- **A tool** is the bridge between an AI's knowledge and its ability to act—the mechanism that transforms a model from a passive information source into an active problem-solver

Characteristics of tool

- **Callable** (arguments)
  - A tool is a function that can be invoked with specific parameters
- **Structured Interface**
  - A tool has a defined interface that the AI understands (JSON)
- **Purpose-Specific**
  - Each tool serves a specific purpose
- **External Capability**
  - A tool provides capabilities the model doesn't have natively
  - Examples
    - Real-time data
    - Computation
    - API Integration
    - Action Execution

Component 1: Tool Definition using tool decorator

- The actual implementation of the tool - the function that does the work

```python
@tool
def tool_name(param1: type, param2: type = default) -> return_type:
    """
    Description: What this tool does and when to use it.

    Args:
        param1: Description of first parameter
        param2: Description of second parameter

    Returns:
        Description of return value
    """
    # Implementation
    result = do_something(param1, param2)
    return result
```

Component 1: Tool Definition using StructuredTool Class



- This method provides a more explicit way to create a tool, often used when you want to define the tool's schema separately from a pre-existing function

```
from langchain_core.tools import StructuredTool

def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

multiply_tool = StructuredTool.from_function(
    func=multiply,
    name="multiply",
    description="Multiply two numbers."
)
```

Component 1: Tool Definition using BaseTool class



- For maximum control, you can directly create a new class that inherits from BaseTool
- This is useful if your tool needs to manage its own state or has very complex execution logic

```python
from langchain_core.tools import BaseTool

class MyCustomCalculatorTool(BaseTool):
    name: str = "multiply"
    description: str = "Multiply two numbers."

    def _run(self, a: int, b: int) -> int:
        # Your custom logic here
        return a * b
```

Component 2: Tool Schema / Binding



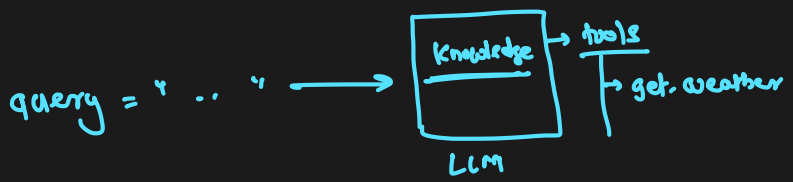
- The structured description of tools that the model can understand. This converts your Python tools into a format the AI can use

```python
# Your Python tool
@tool
def get_weather(city: str) -> str:
    """Get weather for a city."""
    return f"Weather in {city}: 72°F"
```

```python
# LangChain converts it to JSON Schema
{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get weather for a city.",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {
                    "type": "string",
                    "description": "The name of the city"
                }
            },
            "required": ["city"]
        }
    }
}
```

Binding Tools to the Model

- Converts Python tools to JSON schema
- Passes schema to the LLM provider (OpenAI, Anthropic, etc.)
- Trains the model to recognize when to use each tool
- Enables structured output for tool requests
- Provides the model with tool descriptions and parameters




```python
from langchain.chat_models import init_chat_model

# Create model
model = init_chat_model("gpt-4o-mini", model_provider="openai")

# Bind tools to the model
model_with_tools = model.bind_tools([get_weather, calculator])

# Now the model knows about these tools and can request them
```

Component 3: Tool Call Generation

- The model's decision-making process that results in a structured request for a tool

```
# Example: Model's tool call request
response = {
    "content": "",  # Empty when calling tools
    "tool_calls": [
        {
            "id": "call_abc123",  # Unique identifier
            "type": "function",   # Always "function" for now
            "function": {
                "name": "get_weather",  # Which tool to call
                "arguments": '{"city": "Paris"}'  # Parameters as JSON string
            }
        }
    ]
}
```

Component 4: Tool Execution



- The actual running of the tool function with the provided arguments

```python
# Step 1: Parse the tool call
tool_call = response.tool_calls[0]
tool_name = tool_call["function"]["name"]
tool_args = json.loads(tool_call["function"]["arguments"])
tool_id = tool_call["id"]

# Step 2: Look up the tool
tool = tools_by_name[tool_name]  # Dictionary mapping name → tool

# Step 3: Execute the tool
try:
    result = tool.invoke(tool_args)  # Actually runs the function
except Exception as e:
    result = f"Error: {str(e)}"

# Step 4: Format the result
tool_result = {
    "tool_call_id": tool_id,
    "content": result
}
```

```python
# Method 1: Direct invocation
result = get_weather.invoke({"city": "Paris"})

# Method 2: Callable function
result = get_weather("Paris")

# Method 3: Async execution
result = await get_weather.ainvoke({"city": "Paris"})

# Method 4: Batch execution
results = get_weather.batch([
    {"city": "Paris"},
    {"city": "London"}
])
```

Component 5: Result Integration



- The process of taking the tool execution result and feeding it back to the model for final response generation

```
# 1. Append tool result to message history
messages.append({
    "role": "tool",
    "tool_call_id": tool_id,
    "content": tool_result
})

# 2. Model processes the result
final_response = model_with_tools.invoke(messages)

# 3. Model generates final answer
print(final_response.content)
# "The weather in Paris is 72°F and sunny."
```

Benefits of Tool Calling

- Access to Real-Time Information
- Enhanced Computational Capabilities
- Integration with External Systems → API | Database
- Autonomous Decision Making
- Cost and Time Efficiency
- Personalization and Context Awareness
- Enterprise Integration and Business Process Automation
- Data-Driven Decision Making
- Error Reduction and Consistency
- Innovation and New Capabilities

Limitations of Tool Calling



- **Models make mistakes:** They can choose the wrong tool, generate invalid arguments, or even refuse to call a tool when they should
- **Accuracy depends on "soft constraints.":** A model is given a list of tools and told in plain English which ones to use and when
- **Format is fragile:** A tool call must adhere to a specific format. Formatting errors can cause the entire call to fail, and models can struggle to produce consistent output, especially when fine-tuned
- **Performance overhead:** Using frameworks like LangChain can introduce significant latency
- **Framework bugs:** Frameworks themselves can have bugs that break tool calling

Scenarios

Enterprise Customer Support System

- **The Scenario**
  - A large e-commerce company wants to automate their customer support while maintaining high-quality service
  - They need an AI system that can handle complex customer inquiries without human intervention for 80% of cases
- **Real-World Use Case**
  - **Customer:** "I ordered a laptop yesterday but I need to change the shipping address because I'm traveling next week. Also, can you tell me if the laptop comes with a warranty?"

Tool	Purpose	What It Does
Order Lookup	Check order status	Queries the order management system for tracking info, delivery dates, and order details
Refund Processor	Handle returns and refunds	Validates eligibility, calculates refund amounts, processes payments
Inventory Checker	Check product availability	Checks real-time stock levels across multiple warehouses
Shipping Calculator	Calculate delivery times	Estimates shipping costs and delivery dates based on location and method
Customer History	View customer record	Retrieves purchase history, preferences, past interactions
Ticket Creator	Escalate to human agent	Creates support tickets with full conversation context when needed
FAQ Search	Find answers	Searches knowledge base for common questions and solutions
Email Composer	Send follow-ups	Drafts and sends confirmation emails to customers

Healthcare Appointment Management

- **The Scenario**
  - A large hospital network wants to implement an intelligent scheduling and patient management system that can handle everything from appointment booking to pre-visit preparation
- **Real-World Use Case**
  - **Patient:** "I need to see Dr. Smith because I've been having headaches for the past week. I'm also due for my annual physical. Can I get an appointment this week, and what should I do about my blood pressure medication that's almost finished?"

Tool	Purpose	What It Does
Doctor Scheduler	Book appointments	Checks doctor availability, schedules appointments, sends confirmations
Patient Records	Access medical history	Retrieves patient records, medical history, allergies, medications
Insurance Verifier	Check coverage	Validates insurance eligibility and coverage details
Lab Results	Access test results	Retrieves recent lab results and test data
Prescription Checker	Check medications	Verifies current medications and potential interactions
Symptom Analyzer	Assess symptoms	Provides initial symptom assessment and triage recommendations
Appointment Rescheduler	Change appointments	Handles appointment changes, cancellations, and waitlist
Pre-Visit Instructions	Send prep info	Provides tailored preparation instructions for procedures
Medication Refill	Process refills	Handles prescription refill requests

Financial Portfolio Management

- **The Scenario**
  - A wealth management firm wants to provide personalized investment advice and portfolio management to their clients using an AI-powered financial assistant
- **Real-World Use Case**
  - **Client:** "I want to diversify my portfolio. Currently all my money is in tech stocks, and I'm concerned about the upcoming earnings reports. What should I do, and what would be the tax implications if I rebalance?"

Tool	Purpose	What It Does
Stock Price Fetcher	Get real-time prices	Retrieves current stock prices and market data
Portfolio Analyzer	Analyze holdings	Evaluates portfolio performance, diversification, risk metrics
Risk Assessor	Evaluate risk	Assesses risk tolerance and portfolio risk score
Market News	Get news updates	Fetches relevant market news and analyst opinions
Tax Calculator	Estimate taxes	Calculates potential capital gains tax implications
Screener	Find investments	Searches for investment opportunities based on criteria
Trade Executor	Place trades	Executes buy/sell orders (with approval)
Financial Reports	Generate reports	Creates portfolio reports, performance summaries
Currency Converter	Handle FX	Converts between currencies for international investments